



Python Programming Fundamentals

Abstract Classes

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. The abc Module
2. Abstract Shape Class
3. Concrete Subclasses
4. Cannot Instantiate Abstract Class
5. Concrete Method in Abstract Class
6. Using Abstract Classes Polymorphically
7. Abstract Classes vs Regular Classes



Outline

- 1. The abc Module**
2. Abstract Shape Class
3. Concrete Subclasses
4. Cannot Instantiate Abstract Class
5. Concrete Method in Abstract Class
6. Using Abstract Classes Polymorphically
7. Abstract Classes vs Regular Classes



1. The abc Module

```
from abc import ABC, abstractmethod
```

- ABC — Abstract Base Class. Inherit from it to make a class abstract.
- @abstractmethod — decorator that marks a method as **must-override**

A class with **any** @abstractmethod cannot be instantiated.



Outline

1. The abc Module
- 2. Abstract Shape Class**
3. Concrete Subclasses
4. Cannot Instantiate Abstract Class
5. Concrete Method in Abstract Class
6. Using Abstract Classes Polymorphically
7. Abstract Classes vs Regular Classes



2. Abstract Shape Class

```
from abc import ABC, abstractmethod
import math

class Shape(ABC):

    @abstractmethod
    def area(self) -> float:
        """Return the area of this shape."""
        pass

    @abstractmethod
    def perimeter(self) -> float:
        """Return the perimeter of this shape."""
        pass

    def describe(self) -> str:
```



2. Abstract Shape Class

```
return (f"{type(self).__name__}: "  
        f"area={self.area():.2f}, "  
        f"perimeter={self.perimeter():.2f}")
```



Outline

1. The abc Module
2. Abstract Shape Class
- 3. Concrete Subclasses**
4. Cannot Instantiate Abstract Class
5. Concrete Method in Abstract Class
6. Using Abstract Classes Polymorphically
7. Abstract Classes vs Regular Classes



3. Concrete Subclasses

```
class Circle(Shape):
    def __init__(self, radius: float):
        self.__r = radius

    def area(self) -> float:
        return math.pi * self.__r ** 2

    def perimeter(self) -> float:
        return 2 * math.pi * self.__r

class Rectangle(Shape):
    def __init__(self, w: float, h: float):
        self.__w = w
        self.__h = h

    def area(self) -> float:
```



3. Concrete Subclasses

```
    return self.__w * self.__h

def perimeter(self) -> float:
    return 2 * (self.__w + self.__h)
```



Outline

1. The abc Module
2. Abstract Shape Class
3. Concrete Subclasses
- 4. Cannot Instantiate Abstract Class**
5. Concrete Method in Abstract Class
6. Using Abstract Classes Polymorphically
7. Abstract Classes vs Regular Classes



4. Cannot Instantiate Abstract Class

```
# s = Shape()    ← TypeError!  
# TypeError: Can't instantiate abstract class Shape  
# with abstract methods area, perimeter
```

```
c = Circle(5)      # fine – implements both abstract methods  
r = Rectangle(4, 6) # fine
```

If a subclass **forgets** to implement an abstract method:

```
class BadShape(Shape):  
    def area(self):    # only implements one  
        return 0.0  
    # forgot perimeter() !  
  
b = BadShape()
```



4. Cannot Instantiate Abstract Class

```
# TypeError: Can't instantiate abstract class BadShape  
# with abstract method perimeter
```



Outline

1. The abc Module
2. Abstract Shape Class
3. Concrete Subclasses
4. Cannot Instantiate Abstract Class
- 5. Concrete Method in Abstract Class**
6. Using Abstract Classes Polymorphically
7. Abstract Classes vs Regular Classes



5. Concrete Method in Abstract Class

Abstract classes **can** have concrete methods:

```
class Shape(ABC):
    @abstractmethod
    def area(self) -> float: pass

    @abstractmethod
    def perimeter(self) -> float: pass

# Concrete – available to ALL subclasses
def describe(self) -> str:
    return (f"{type(self).__name__}: "
            f"area={self.area():.2f}")

def is_larger_than(self, other: "Shape") -> bool:
    return self.area() > other.area()
```



Outline

1. The abc Module
2. Abstract Shape Class
3. Concrete Subclasses
4. Cannot Instantiate Abstract Class
5. Concrete Method in Abstract Class
- 6. Using Abstract Classes Polymorphically**
7. Abstract Classes vs Regular Classes



6. Using Abstract Classes Polymorphically

```
shapes: list[Shape] = [  
    Circle(5),  
    Rectangle(4, 6),  
    Circle(3),  
    Rectangle(10, 2),  
]  
  
# All are Shapes – can call area() and perimeter() safely  
for shape in shapes:  
    print(shape.describe())  
  
total = sum(s.area() for s in shapes)  
print(f"\nTotal area: {total:.2f}")  
  
largest = max(shapes, key=lambda s: s.area())  
print(f"Largest: {largest.describe()}")
```



Outline

1. The abc Module
2. Abstract Shape Class
3. Concrete Subclasses
4. Cannot Instantiate Abstract Class
5. Concrete Method in Abstract Class
6. Using Abstract Classes Polymorphically
- 7. Abstract Classes vs Regular Classes**



7. Abstract Classes vs Regular Classes

Abstract Class	Regular Class
Cannot be instantiated	Can be instantiated
Forces subclasses to implement methods	No enforcement
Defines an interface/contract	Defines concrete behaviour
Inherits from ABC	Inherits from object
Has @abstractmethod methods	All methods have implementations

Thanks for Watching - Any questions?

